

DashyDashboard

Production Deployment Guide

ASP.NET 7 (**net7.0**) | IIS | Windows Authentication | SQL Server

Target SQL Server: clipvwbpod02 | Database: ClientsAppAttestation

June 2026

Contents

1	Do This First	2
2	What to Install and Where	2
3	Exactly Which File Do I Edit?	3
4	What Do I Paste Into It?	4
5	If I Want to Set the Connection String in IIS Instead	5
6	Windows Authentication Setup	6
7	Build and Publish	8
8	How to Keep Demo Data Out of Production	9
9	What Remains Before First Production Run	10

Do This First

Work through these steps in order. Each step has a section below that tells you exactly what to do.

1. **Read Section 9 right now.** There are known database problems that will corrupt data if you go live too early.
2. Install the right software on the build machine and on the IIS server (Section 2).
3. Edit the connection string file in the source repository, *before* you publish (Sections 3 and 4).
4. Run the one-line publish command on the build machine (Section 7, Step 3).
5. Copy the published output folder to the IIS server (Section 7, Step 4).
6. Run database migrations from a machine that can reach `clipvwbpod02` (Section 7, Step 5).
7. Create the IIS application pool and website, enable Windows Authentication, disable Anonymous Authentication (Section 6).
8. Confirm all three `DeveloperMode` flags are `false` (Section 8).

STOP before you point this at the live database. Read Section 9 first. Known schema mismatches in the current code will cause data corruption and leave Admin/GFH/IFH role enforcement completely bypassed.

What to Install and Where

Two machines are involved. The **build machine** is where you compile and publish the code. The **IIS server** is the Windows Server that users browse to. They need different software.

Software	Build machine	IIS server
.NET SDK 7.0.x or later	Required. The app targets <code>net7.0</code> . SDK 7.0.x is the minimum. SDK 8.0.x also builds <code>net7.0</code> projects and is acceptable.	Not needed.
.NET 7 Hosting Bundle	Not needed.	Required. Installs the ASP.NET Core 7 runtime and the IIS module that runs the app.
Node.js 18 or later	Required. The publish command runs <code>npm ci</code> and <code>npm run build</code> automatically.	Not needed at all. React is compiled into static files during publish.
IIS	Not needed.	Required.

SDK vs Runtime. The project file targets `net7.0`. The **server** only ever needs the .NET 7 runtime (the Hosting Bundle). The **build machine** needs the .NET 7 SDK (minimum); SDK 8.x also builds `net7.0` projects so either works. **Important:** .NET 7 is out of Microsoft mainstream support. Plan an upgrade to .NET 8 or .NET 9 in a future sprint.

Install .NET SDK 7.0.x on the build machine

1. Open a browser. Go to <https://dotnet.microsoft.com/en-us/download/dotnet/7.0>
2. Under **SDK**, click the **Windows x64 Installer** link. Download the file.

3. Double-click the downloaded `.exe` file. A Windows security prompt appears. Click **Yes**.
4. Click **Install**. Wait for the progress bar to finish. Click **Close**.
5. Open a **new** Command Prompt window. (Press **Win + R**, type `cmd`, press **Enter**.) Run:

```
dotnet --version
```

The output must begin with 7. (or 8. if you have SDK 8 installed — both work).

Install Node.js 18 LTS on the build machine

1. Open a browser. Go to <https://nodejs.org/en/download>
2. Click the **Windows Installer (.msi)** link for version **18.x LTS**. Download the file.
3. Double-click the `.msi` file. Click **Next** through every screen. Accept the licence. Leave all options at their defaults. Click **Install**. Accept the UAC prompt. Click **Finish**.
4. Open a **new** Command Prompt window. Run:

```
node --version
```

The output must begin with `v18`.

Install the .NET 7 Hosting Bundle on the IIS server

Do this on the **IIS server**, not on the build machine. Log in to the IIS server directly or via Remote Desktop.

1. Open a browser on the IIS server. Go to <https://dotnet.microsoft.com/en-us/download/dotnet/7.0>
2. Under **ASP.NET Core Runtime 7.0.x**, click the **Hosting Bundle** link. Download the file.
3. Double-click the installer. Click **Install**. Accept the UAC prompt. Wait for it to finish. Click **Close**.
4. Open a Command Prompt as **Administrator**: click **Start**, type `cmd`, right-click **Command Prompt**, click **Run as administrator**. Run:

```
dotnet --list-runtimes
```

You should see a line containing `Microsoft.AspNetCore.App 7.0`.

5. Still as Administrator, run:

```
iisreset
```

IIS must be restarted so it loads the new ASP.NET Core module.

Exactly Which File Do I Edit?

Before publishing (working inside the source repository)

Open this file in any text editor — Notepad, Notepad++, or VS Code. The path below is relative to the root of the repository:

```
DashyDashboard/DashyDashboard.Api/appsettings.Production.json
```

Edit this file *before* you run the publish command. The publish step copies this file into the output folder automatically.

After publishing (on the IIS server)

If you have already published and just need to update the connection string without republishing, edit the file at the physical path of the IIS site:

```
<IIS site physical path>\appsettings.Production.json
```

For example, if your IIS site's physical path is `C:\inetpub\DashyDashboard`, the file you edit is:

```
C:\inetpub\DashyDashboard\appsettings.Production.json
```

To open it: open File Explorer, navigate to `C:\inetpub\DashyDashboard`, right-click `appsettings.Production.json`, click **Open with**, click **Notepad**.

How the app decides which settings file to load

The app loads files in this order. Values in later entries override earlier ones.

1. `appsettings.json` — base file, always loaded, contains no connection string
2. `appsettings.Production.json` — loaded only when `ASPNETCORE_ENVIRONMENT=Production`; contains the connection string
3. Environment variables — override both files; can be set in IIS Manager or in `web.config`

If `appsettings.Production.json` is missing or the connection string is blank, the app throws this error at startup and refuses to start: *“Connection string ‘Default’ is not configured. Set it in appsettings.Production.json or via environment variables.”*

Tell the app it is running in Production

The app must know it is in the Production environment or it will not load the `appsettings.Production.json` file. Set the `ASPNETCORE_ENVIRONMENT` variable in the published `web.config`.

1. On the IIS server, open File Explorer. Navigate to the site folder, e.g. `C:\inetpub\DashyDashboard`.
2. Right-click `web.config`. Click **Open with**. Click **Notepad**.
3. Find the line that starts with `<aspNetCore`. Look for an `<environmentVariables>` block inside it. If that block does not exist, add it. The result should look exactly like this:

```
<aspNetCore processPath=".\\DashyDashboard.Api.exe"
  stdoutLogEnabled="false"
  stdoutLogFile=".\logs\stdout"
  hostingModel="inprocess">
  <environmentVariables>
    <environmentVariable
      name="ASPNETCORE_ENVIRONMENT"
      value="Production" />
  </environmentVariables>
</aspNetCore>
```

4. Press **Ctrl + S** to save. Close Notepad. IIS picks up `web.config` changes immediately — no restart needed.

What Do I Paste Into It?

Open `appsettings.Production.json` in Notepad. Press **Ctrl + A** to select everything. Press **Delete** to clear the file. Now paste this block exactly as it appears below:

```
{
  "ConnectionStrings": {
    "Default": "Server=clipvwbpod02;Database=ClientsAppAttestation;Integrated
    ↪ Security=True;Encrypt=True;TrustServerCertificate=False;"
  },
  "DeveloperMode": {
    "EnableFormLogin": false,
    "EnableUserHeaderAuth": false,
    "EnableDemoSeedData": false
  },
  "AllowedHosts": "*"
}
```

Press **Ctrl + S** to save the file. That is the entire file.

What each part means:

- **Server=clipvwbpod02** — The hostname of the production SQL Server. Do not change this unless a DBA tells you to.
- **Database=ClientsAppAttestation** — The database name. Do not change this.
- **Integrated Security=True** — Connect to SQL Server using the Windows identity of the IIS application pool. No username or password is stored in this file.
- **Encrypt=True** — All traffic between the app and the database is encrypted. Required in production.
- **TrustServerCertificate=False** — The SQL Server's certificate must be valid. Do not change this to **True** in production.
- **EnableFormLogin: false** — Disables the developer-only form login endpoint. Must be **false**.
- **EnableUserHeaderAuth: false** — Disables the **X-User-Id** header shortcut that bypasses Windows auth. Must be **false**.
- **EnableDemoSeedData: false** — Prevents fictional test users from being inserted into the database. Must be **false**.

Never change any DeveloperMode value to true in production. **EnableFormLogin** and **EnableUserHeaderAuth** bypass Windows Authentication and allow any caller to impersonate any user without a password. **EnableDemoSeedData** will insert Bruce Banner, Diana Prince, Selina Kyle, and ≈625 fictional test records into the live database.

If I Want to Set the Connection String in IIS Instead

If your deployment pipeline overwrites `appsettings.Production.json` on every deploy, set the connection string as an environment variable in IIS. Environment variables always override the settings file.

Method A — IIS Manager (no file editing needed)

1. Press **Win + R**, type `inetmgr`, press **Enter**. IIS Manager opens.
2. In the left panel, expand the server name, expand **Sites**, and click on **DashyDashboard**.
3. In the centre panel, double-click **Configuration Editor**.
4. At the top of the Configuration Editor, find the **Section** dropdown. Click it and navigate to `system.webServer → aspNetCore`.
5. In the table that appears in the centre, find the row labelled **environmentVariables**. Click the grey `[...]` button at the right end of that row. A Collection Editor window opens.
6. In the top-right of the Collection Editor, click **Add Item**. A new blank row appears in the list.
7. Click the **name** cell of that new row and type:

```
ConnectionStrings__Default
```

(Two underscores between `ConnectionStrings` and `Default`.)

8. Click the **value** cell and type:

```
Server=clipvwbpod02;Database=ClientsAppAttestation;Integrated Security=True;
↳ Encrypt=True;TrustServerCertificate=False;
```

9. Click **Apply** in the top-right Actions panel. Close the Collection Editor.
10. Click **Apply** again in the main Configuration Editor panel.
11. In the left panel, click **Application Pools**. Right-click **DashyDashboard**. Click **Recycle**.

Method B — Edit web.config directly

Open `C:\inetpub\DashyDashboard\web.config` in Notepad. Find the `<aspNetCore>` element. Add both variables inside an `<environmentVariables>` block:

```
<aspNetCore processPath=".\\DashyDashboard.Api.exe"
  stdoutLogEnabled="false"
  stdoutLogFile=".\logs\stdout"
  hostingModel="inprocess">
  <environmentVariables>
    <environmentVariable
      name="ASPNETCORE_ENVIRONMENT"
      value="Production" />
    <environmentVariable
      name="ConnectionStrings__Default"
      value="Server=clipvwbpod02;Database=ClientsAppAttestation;Integrated Security
      ↪ =True;Encrypt=True;TrustServerCertificate=False;" />
  </environmentVariables>
</aspNetCore>
```

Press **Ctrl + S**. No IIS restart needed. IIS applies `web.config` changes immediately.

Why two underscores? ASP.NET Core converts environment variable names to configuration keys by replacing `__` (double underscore) with `:` (colon). So the variable `ConnectionStrings__Default` maps to the key `ConnectionStrings:Default`, which is exactly the key the app reads.

Windows Authentication Setup

The app uses Windows/Negotiate authentication. IIS identifies the user from their Windows login and passes that identity to the app. You must enable Windows Authentication and disable Anonymous Authentication.

Step 1 — Enable the Windows Authentication feature in Windows Server

Do this once. If Windows Authentication is already listed in IIS, skip to Step 2.

1. Click **Start**. Type **Server Manager**. Press **Enter**.
2. Click **Manage** in the top-right corner. Click **Add Roles and Features**.
3. Click **Next** through the first three pages (Before You Begin, Installation Type, Server Selection).
4. On **Server Roles**, find and expand: **Web Server (IIS)** → **Web Server** → **Security**.
5. Tick the checkbox next to **Windows Authentication**. If a dialog says “Add features that are required?”, click **Add Features**.
6. Click **Next** twice more, then click **Install**.
7. Wait for the installation bar to finish. Click **Close**.

Step 2 — Create the application pool

1. Press **Win + R**. Type `inetmgr`. Press **Enter**.
2. In the left panel, click **Application Pools**.
3. In the right-side Actions panel, click **Add Application Pool...**
4. In the dialog that appears, fill in:
 - **Name:** `DashyDashboard`
 - **.NET CLR version:** `No Managed Code` (this is correct — the app is self-contained)
 - **Managed pipeline mode:** `Integrated`

5. Click **OK**.

Step 3 — Create the website

1. In the left panel, right-click **Sites** and click **Add Website...**
2. Fill in every field:
 - **Site name:** DashyDashboard
 - **Application pool:** click **Select...** and pick DashyDashboard
 - **Physical path:** the folder on this server where you copied the published output, for example C:\inetpub\DashyDashboard
 - **Binding type:** https
 - **IP address:** All Unassigned
 - **Port:** 443
 - **Host name:** the DNS name users will type in their browser, for example `dashy.broadridge.internal`
 - **SSL certificate:** pick the certificate that matches your host name from the dropdown
3. Click **OK**.

The application server hostname (what users type in the browser) is not stored anywhere in the repository. It is whatever DNS name you enter in the Host name field above. Tell users to browse to that name.

Step 4 — Configure authentication

1. In the left panel, click on your **DashyDashboard** site.
2. In the centre panel, double-click **Authentication**.
3. Click **Anonymous Authentication** once (just to select it). In the right Actions panel, click **Disable**.
4. Click **Windows Authentication** once to select it. In the right Actions panel, click **Enable**.
5. With **Windows Authentication** still selected, click **Providers...** in the right Actions panel. A small dialog opens showing a list.
6. The list must show **Negotiate** above **NTLM**. If it does not, click **Add**, select **Negotiate**, click **OK**, then use the **Move Up** button to put Negotiate above NTLM.
7. Click **OK** to close the Providers dialog.

The authentication panel should now show:

Authentication type	Status
Anonymous Authentication	Disabled
Windows Authentication	Enabled

Step 5 — Grant the app pool identity access to SQL Server

The app connects to SQL Server using the Windows account of the IIS application pool. Ask the DBA to run this SQL on `clipvwbpod02`:

```
USE ClientsAppAttestation;

CREATE LOGIN [IIS APPPOOL\DashyDashboard]
  FROM WINDOWS
  WITH DEFAULT_DATABASE = ClientsAppAttestation;

CREATE USER [IIS APPPOOL\DashyDashboard]
  FOR LOGIN [IIS APPPOOL\DashyDashboard];

ALTER ROLE db_datareader
```

```
ADD MEMBER [IIS APPPOOL\DashyDashboard];

ALTER ROLE db_datawriter
ADD MEMBER [IIS APPPOOL\DashyDashboard];
```

If a domain service account is used instead of the default pool identity, replace every occurrence of IIS APPPOOL\DashyDashboard with that account name.

Build and Publish

All commands in this section are run on the **build machine**, not the IIS server.

Step 1 — Open a Command Prompt at the repo root

1. Press **Win + R**. Type `cmd`. Press **Enter**.
2. Type `cd` followed by a space, then the full path to the repository root. Press **Enter**. For example:

```
cd C:\projects\dashythedashdashboard
```

Replace `C:\projects\dashythedashdashboard` with wherever the repository actually lives on your machine.

Step 2 — Confirm prerequisites

```
dotnet --version
```

Output must begin with 7. or higher (e.g. 7.0.407 or 8.0.421).

```
node --version
```

Output must begin with v18. or higher.

If either is wrong or missing, go back to Section 2 and install.

Step 3 — Edit the connection string (if you have not already)

Follow Sections 3 and 4. Save the file before continuing.

Step 4 — Run the publish command

Type this command exactly and press **Enter**:

```
dotnet publish DashyDashboard/DashyDashboard.Api/DashyDashboard.Api.csproj -c Release
↵ -o publish/DashyDashboard.Api
```

You do not need to run any npm or frontend commands manually. The publish step does all of the following automatically:

1. Runs `npm ci` inside `DashyDashboard/DashyDashboard.Frontend` to install all Node packages.
2. Runs `npm run build` to compile the React app into static files.
3. Copies those static files into the API's `wwwroot` folder.
4. Compiles the .NET API in Release mode.
5. Copies every file needed to run the application into `publish\DashyDashboard.Api`.

When it finishes successfully, the last line of output will be:

```
Build succeeded.
```

If you see an error about `npm` not being found, install Node.js 18 (Section 2) and try again.

Step 5 — Copy the output to the IIS server

The published output is now in the folder `publish\DashyDashboard.Api` on the build machine. Copy everything inside that folder to the site physical path on the IIS server.

Run this command from the repository root (replace `your-iis-server` with the actual server name or IP address):

```
robocopy publish\DashyDashboard.Api \\your-iis-server\c$\inetpub\DashyDashboard /MIR
↪ /Z
```

Alternatively, copy the folder manually using Windows File Explorer over a network share.

Step 6 — Run database migrations

Do not skip this. In production, the app does not apply migrations automatically at startup. If you skip this step, the tables will not exist and every API call will fail.

Run this from a machine that: (a) has the .NET 7 SDK (or later) installed, and (b) can reach `clipvwbpod02` on the network (this can be the build machine if it has network access to the database server).

In the same Command Prompt, navigate to the API project:

```
cd DashyDashboard/DashyDashboard.Api
```

Check that the EF Core CLI is installed:

```
dotnet ef --version
```

If that command prints “command not found” or fails, install it:

```
dotnet tool install --global dotnet-ef
```

Then run the migrations:

```
dotnet ef database update --connection "Server=clipvwbpod02;Database=
↪ ClientsAppAttestation;Integrated Security=True;Encrypt=True;
↪ TrustServerCertificate=False;"
```

You will see each migration name printed one by one. The last line will be:

```
Done.
```

If you get an SSL certificate error, ask the DBA to confirm that the SQL Server certificate on `clipvwbpod02` is valid and trusted by the machine you are running from. If the DBA confirms the cert is internal-only, change `TrustServerCertificate=False` to **True only for this migration command**, not in the deployed settings file.

How to Keep Demo Data Out of Production

The application includes seed data with fictional users: **Bruce Banner**, **Diana Prince**, **Selina Kyle**, and approximately 625 additional test records.

The seed code runs **only when both** of these conditions are true simultaneously:

- `ASPNETCORE_ENVIRONMENT` equals `Development`, and
- `DeveloperMode:EnableDemoSeedData` equals `true`

In production, neither condition is true if you follow this guide. The `appsettings.Production.json` from Section 4 already sets `EnableDemoSeedData` to `false`, and `ASPNETCORE_ENVIRONMENT` is `Production`.

Verification checklist — check each of these before go-live:

1. Open `C:\inetpub\DashyDashboard\appsettings.Production.json` in Notepad. Confirm the `DeveloperMode` block reads exactly:

```
"DeveloperMode": {
  "EnableFormLogin": false,
  "EnableUserHeaderAuth": false,
  "EnableDemoSeedData": false
}
```

All three values must be `false` (no quotes around it, not `"false"`, not missing).

2. Open IIS Manager. Click on the site. Double-click **Configuration Editor**. Navigate to `system.webServer/aspNetCore`. Click [...] next to **environmentVariables**. Confirm there is no variable named `DeveloperMode__EnableDemoSeedData` with value `true`. Environment variables override the file.
3. Confirm the variable `ASPNETCORE_ENVIRONMENT` is set to `Production` (not `Development`) in either `web.config` or the IIS Configuration Editor.

What Remains Before First Production Run

Schema alignment is complete. All blockers from the original pre-development audit have been resolved in the current codebase: `AssociateID` is `varchar(50)` throughout; `ToolID` is `int` throughout; all ten spec tables exist (`SuperUsers`, `IFHGFHMapping`, `AttestationLogs`, `LoginLogs`, and the rest); login attempts are written to `LoginLogs` on every request. The build is clean (0 errors).

What remains is operational, not code.

Mandatory — Run EF Core migrations

Covered in Section 7, Step 6. The migrations create all ten tables. The database starts empty after migration — no application data is inserted.

Mandatory — Import production data from Excel

The application tables are populated from the authoritative Excel files maintained by the operations team. The source files referenced in the schema documentation are:

- `C:\Prakash Files\Dash\Users.xlsx` → `dbo.Users`
- `C:\Prakash Files\Dash\Clients.xlsx` → `dbo.Clients`
- `C:\Prakash Files\Dash\ClientTools.xlsx` → `dbo.ClientTools`
- `C:\Prakash Files\Dash\UsersToolAccess.xlsx` → `dbo.UsersToolAccess`
- `C:\Prakash Files\Dash\Department IFH GFH Mapping.xlsx` → `dbo.IFHGFHMapping`

Use the `OPENROWSET / ACE OLEDB` import pattern from Appendix A of the schema PDF (`ClientsAppAttestation_Table_Structure_Documentation.pdf`). Enable Ad Hoc Distributed Queries on the SQL Server first:

```
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'Ad Hoc Distributed Queries', 1; RECONFIGURE;
```

The SQL Server service account on `clipvwbpod02` must be able to read the file path above (either a local path on the SQL Server machine, or a UNC path accessible from it).

Mandatory — Populate SuperUsers for the go-live team

Once Users are imported, insert a row in `dbo.SuperUsers` for each person who needs Admin, VP, GFH, or IFH access. Ask the DBA to run:

```
USE ClientsAppAttestation;

INSERT INTO dbo.SuperUsers
    (AssociateID, RoleName, Department, AccessLevel, IsActive, CreatedBy)
VALUES
    ('173926', 'Admin', 'DTC Settlements', 'Full', 'TRUE', 'System'),
    ('179335', 'VP', 'International', 'Approver', 'TRUE', 'System');
-- Add one row per admin/VP/GFH/IFH user.
```

Use the real `AssociateID` values (VARCHAR, e.g. '173926') and the real `RoleName` values: Admin, VP, GFH, or IFH.

Critical — Set the `UserName` column for Windows Authentication

This step is the most common reason users reach the app but see nothing.

When a user opens the app in their browser, IIS identifies them via Windows Authentication (Kerberos/NTLM) and passes their identity to the application as `HttpContext.User.Identity.Name`. The app then queries:

```
SELECT * FROM dbo.Users WHERE UserName = '<value IIS sent>';
```

If no row matches, the user is unresolved and the app behaves as if nobody is logged in.

What value does IIS send? On most corporate Windows setups, `User.Identity.Name` is `DOMAIN\loginname`, for example `BROADRIDGE\KalapurayilP`. The `UserName` column in `dbo.Users` must contain exactly that string for each user.

Verify what IIS sends on your network by enabling stdout logging temporarily (see the Troubleshooting Guide, Section 2) and looking for the identity in the application logs, or ask the infrastructure team what domain prefix IIS uses.

Once confirmed, update the column in bulk:

```
-- Example: if IIS sends BROADRIDGE\<loginname>
UPDATE dbo.Users
SET UserName = 'BROADRIDGE\' + UserName
WHERE UserName NOT LIKE 'BROADRIDGE\%';
```

If IIS sends the bare login name without a domain prefix, the `UserName` values already imported from Excel (e.g. `KalapurayilP`) will match directly and no update is needed.

Remaining non-blocking limitations

- **No mobile layout.** The frontend targets desktop viewports (1280px and wider).
- **Dev form login accepts any password.** The `POST /api/auth/login` endpoint is disabled in Production (`EnableFormLogin: false`) but in Development it accepts any non-empty password without verification. This is intentional — Production never uses form login.
- **ClientTools and UsersToolAccess use the import-table structure.** `dbo.ClientTools` uses `UserID int identity` as its PK and `[Application Name] varchar(255)` for the tool name, matching the flat Excel import design. The normalised design (composite key on `ClientID+ToolName`) is enforced as a unique constraint but not the primary key. This is intentional and matches the production database.